

✓ STEP 1: Represent Graph (Adjacency List)

```
from collections import deque

graph = {
    "Islamabad": ["Rawalpindi", "Lahore", "Peshawar"],
    "Rawalpindi": ["Islamabad", "Peshawar", "Quetta"],
    "Peshawar": ["Islamabad", "Rawalpindi", "Quetta"],
    "Lahore": ["Islamabad", "Multan", "Quetta"],
    "Multan": ["Lahore", "Karachi", "Quetta"],
    "Quetta": ["Rawalpindi", "Peshawar", "Multan", "Karachi"],
    "Karachi": ["Multan", "Quetta"]
}
```

✓ STEP 2: BFS Algorithm to Find Shortest Path

```
def bfs_shortest_path(graph, start, goal):
    queue = deque([[start]])
    visited = set()

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)

    return None
```

✓ STEP 3: Run BFS

```
path = bfs_shortest_path(graph, "Islamabad", "Karachi")
print("Shortest path:", " → ".join(path))
print("Number of cities:", len(path))
```

```
Shortest path: Islamabad → Rawalpindi → Quetta → Karachi
Number of cities: 4
```

